

Curso: Go e System Design para processamento de arquivos em alto volume

Oct 7, 2026 · @Daniel Freire

Como usar este curso

Você vai sair daqui sabendo projetar e implementar, em Go 1.27, um pipeline que recebe milhares de arquivos por dia, processa milhões de linhas e grava tudo no PostgreSQL sem estourar memória, sem duplicar dados e sem perder registros quando algo falha.

É o equivalente direto do curso de .NET 10 sobre o mesmo tema: os mesmos dez módulos, o mesmo método **top-down** (o problema e os números, depois a arquitetura, depois cada componente, só então o código) e o mesmo projeto final. A parte independente de linguagem (requisitos, Lei de Little, arquitetura, SQL, particionamento, idempotência) é resumida aqui e aprofundada no curso de .NET, com referência ao módulo exato. O espaço economizado vai para o que é diferente em Go: um GC sem gerações, `[]byte` no lugar de `Span<T>`, canais e goroutines no lugar de `Channel<T>` e tarefas, `errgroup` no lugar de `Task.WhenAll`, e o `CopyFrom` do pgx no lugar do `COPY` do Npgsql.

Pré-requisitos

Sintaxe de Go, incluindo goroutines, canais, `select` e `context.Context`. Se você fez o curso de APIs HTTP em Go, já tem tudo, inclusive pgx, slog e OpenTelemetry. Não é preciso ter feito o curso de .NET, mas os dois juntos mostram o mesmo problema resolvido em dois ecossistemas.

Exemplo condutor

Os mesmos arquivos de retorno bancário em layout posicional (CNAB 400), com linhas de largura fixa, header e trailer com totais de controle, chegando em rajadas e precisando ser processados exatamente uma vez. As tabelas são as mesmas do curso de .NET (`arquivo`, `retorno_detalhe`, `outbox`), e a API em Go do curso de APIs recebe os arquivos que este worker processa.

Stack usada

- Go 1.27 e a biblioteca padrão (`bufio`, `bytes`, `crypto/sha256`, `log/slog`, `uuid`, `testing`)
- `golang.org/x/sync/errgroup` para orquestrar goroutines com erro e cancelamento
- PostgreSQL com `pgx` (incluindo `CopyFrom`) e `sqlc` para as consultas de controle

- AWS SDK para Go v2 (S3 e SQS); os conceitos valem para qualquer storage e fila
- OpenTelemetry para Go, `pprof` e o pacote `testing` para benchmarks

Do .NET para Go, neste problema

Peça	Curso de .NET 10	Este curso, Go 1.27
Memória sem cópia	<code>Span<T></code> , <code>ReadOnlySpan<byte></code>	Fatias de <code>[]byte</code> sobre o buffer de leitura
Reuso de buffers	<code>ArrayPool<T></code>	Canal de buffers livres ou <code>sync.Pool</code>
Leitura em streaming	<code>System.IO.Pipelines</code>	<code>bufio.Scanner</code> / <code>bufio.Reader</code>
Fila dentro do processo	<code>Channel<T></code> com capacidade	Canal com buffer (<code>make(chan T, n)</code>)
Orquestração de estágios	<code>Task.WhenAll</code> + <code>CancellationTokenSource</code>	<code>errgroup.WithContext</code>
Carga no banco	<code>BeginBinaryImportAsync</code> (Npgsql)	<code>CopyFrom</code> (pgx)
Retry	Polly	Função própria de backoff com jitter
Host do worker	<code>BackgroundService</code>	<code>main</code> + <code>signal.NotifyContext</code>
GC	Geracional, com LOH	Concorrente, sem gerações, controlado por <code>GOGC</code> e <code>GOMEMLIMIT</code>

Mapa dos módulos

Módulo	Pergunta que responde	O que você entrega
1. O problema	Quanto volume, com que prazo, com que garantias?	A ficha de requisitos com números
2. Arquitetura	Quais peças existem e como conversam?	O diagrama de referência
3. Runtime	O que o GC do Go cobra e como não pagar?	Um parser sem alocação por linha

Módulo	Pergunta que responde	O que você entrega
4. I/O de arquivos	Como ler 2 GB sem carregar 2 GB?	Leitura em streaming do disco e do S3
5. Concorrência	Como paralelizar sem afogar o banco?	O pipeline com canais, errgroup e buffers reaproveitados
6. Escrita em massa	Como inserir 1 milhão de linhas em segundos?	Um escritor com <code>CopyFrom</code> e <code>upsert</code>
7. Leitura em massa	Como ler tabelas grandes sem travar nada?	Consultas em streaming, keyset e <code>COPY TO</code>
8. Confiabilidade	O que acontece quando algo quebra no meio?	Lease, retries, DLQ, outbox e reconciliação
9. Observabilidade	Como sei que está rápido e saudável?	Métricas, traces, perfis e benchmarks
10. Escala	Como crescer de 1 para N workers?	Deploy com autoscaling por fila
Projeto final	Tudo junto funciona?	O worker completo, testado sob carga e caos

Sugestão de ritmo: um módulo por semana. Os Módulos 3, 4 e 5 concentram o que mais difere do .NET; o 5 é onde Go mais brilha.

Módulo 1 — O problema: volume, prazos e garantias

Antes do código, três números: quantas linhas por segundo o sistema sustenta no pico, em quanto tempo um arquivo precisa estar processado e qual garantia de entrega o negócio exige. Esses números não dependem da linguagem, e o Módulo 1 do curso de .NET os desenvolve em detalhe. Aqui eles são resumidos e complementados com o que muda quando o worker é escrito em Go.

1.1 Onde o jeito ingênuo quebra

O jeito ingênuo em Go é `os.ReadFile` (o arquivo inteiro num `[]byte`), `strings.Split` por linha (uma string por linha) e um `INSERT` por registro. Ele quebra nos mesmos três pontos do .NET:

1. **Memória:** o arquivo inteiro na memória, mais uma cópia em strings. Em Go strings são UTF-8 (não dobram de tamanho como no .NET), mas cada linha ainda é uma alocação com cabeçalho, e o GC precisa percorrer todas.

2. **Banco:** um `INSERT` por linha é uma ida e volta de rede por linha. Com 1 ms de latência, 1 milhão de linhas custam mais de 16 minutos só de espera.
3. **Falha:** se o processo cai no meio, não se sabe o que foi gravado.

1.2 Os números de exemplo

O mesmo cenário do curso de .NET, para que os dois cursos sejam comparáveis:

Medida	Valor	Como chegar nele
Arquivos por dia	5.000	Dado do negócio
Linhas por arquivo	40.000 em média; 3 milhões no maior	Dado do negócio
Volume diário	200 milhões de linhas	5.000×40.000
Pico	60% em 2 horas: cerca de 16.700 linhas/s	$120 \text{ milhões} \div 7.200 \text{ s}$
Alvo de projeto	cerca de 35.000 linhas/s	Pico com margem de 2×
SLA do maior arquivo	15 minutos: pelo menos 3.300 linhas/s num só arquivo	$3 \text{ milhões} \div 900 \text{ s}$
Dados por dia	cerca de 60 GB sem índices	$200 \text{ milhões} \times 300 \text{ bytes}$

A última linha já obriga a pensar em particionamento e retenção (Módulo 7), exatamente como no .NET.

1.3 Lei de Little e gargalos

A relação entre concorrência, vazão e latência vale para goroutines como valia para tarefas:

$$L = \lambda \cdot W$$

Com 35.000 linhas/s em lotes de 5.000 (7 lotes por segundo) e 0,5 s por lote no banco, são 3,5 lotes em voo ao mesmo tempo: pelo menos 4 escritores. Se o banco ficar mais lento e cada lote levar 1 s, são 7.

Os três tipos de gargalo também são os mesmos: I/O de rede ou disco (o processo espera), CPU (parsing e validação) e banco (o worker espera o banco). Em pipelines de arquivo, quase sempre é o banco, e o design gira em torno de protegê-lo: lotes, `COPY`, concorrência limitada e back-pressure.

1.4 Garantias de entrega

A equação central do curso de .NET continua sendo a central aqui: **pelo menos uma vez + idempotência = efeito de exatamente uma vez**. A fila e os retries garantem que nada se perde; as chaves naturais e o `ON CONFLICT` garantem que repetir não duplica. O Módulo 8 constrói isso camada por camada.

1.5 O que muda quando o worker é em Go

- **Concorrência barata e explícita.** Uma goroutine custa alguns KB; um pipeline com dezenas delas é normal. E elas não são assíncronas no sentido do .NET: o código é bloqueante e direto, e o runtime troca de goroutine quando uma espera I/O. Não existe `async / await`, nem o risco de bloquear o ThreadPoo com `.Result`.
- **Paralelismo real em CPU.** O mesmo código bloqueante roda em paralelo em todos os núcleos disponíveis, sem `Task.Run`. Se o parsing virar gargalo, dois leitores sobre chunks diferentes usam dois núcleos.
- **Um GC diferente.** O GC do Go não tem gerações nem LOH: é concorrente, não move objetos e é regulado pelo crescimento do heap. Alocação por linha continua sendo o inimigo, mas pelo motivo explicado no Módulo 3.
- **Erros como valores e cancelamento por contexto.** Cada estágio devolve `error`, e o `errgroup` cancela os outros quando um falha. É o mesmo desenho do `Estagio` com `CancellationTokenSource` do curso de .NET, só que com uma biblioteca de dez linhas no lugar do código de orquestração.
- **Um binário estático.** O worker inteiro é um executável sem dependências, numa imagem de poucas dezenas de MB, que sobe em milissegundos. Isso pesa a favor no autoscaling (Módulo 10).

Exercício do Módulo 1

Preencha a ficha de requisitos do seu cenário real (ou use a tabela 1.2): arquivos por dia, linhas médias e máximas, janela de pico, SLA, garantia de entrega e retenção. Calcule linhas por segundo no pico, volume diário e número de escritores pela Lei de Little. Guarde a ficha: o projeto final é validado contra ela, e ela é a mesma que serviria para a versão em .NET.

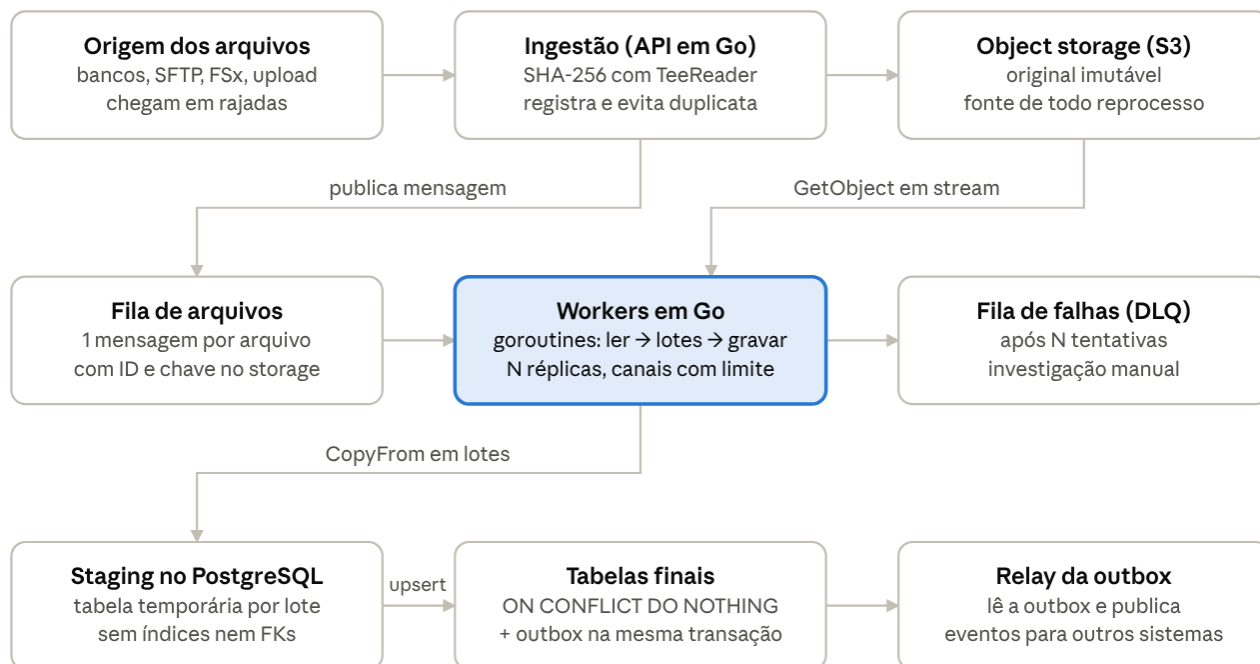
Módulo 2 — System design: a arquitetura de referência

A arquitetura é a mesma do curso de .NET, porque ela resolve um problema de sistema, não de linguagem: separar **receber** de **processar**, guardar o original intacto, distribuir o trabalho por uma fila, processar em workers sem estado que escrevem em lotes numa

área de staging e consolidam de forma idempotente, e propagar eventos pela outbox. Em Go, a ingestão é a API do curso de APIs HTTP, e o worker é o assunto deste curso.

O arquivo entra uma vez, é processado em lotes e vira eventos pela outbox

Arquitetura de referência em Go · o worker em destaque é o foco dos Módulos 3 a 7



arquitetura de referência · 9 componentes, 3 camadas

Leia de cima para baixo: a primeira linha recebe e guarda, a segunda distribui o trabalho, a terceira grava e propaga. Cada seta é um ponto de falha possível, e cada componente foi escolhido para que essa falha seja recuperável.

2.1 Os seis princípios

1. **Separe ingestão de processamento**, para que uma rajada seja aceita em segundos e drenada no ritmo do banco.
2. **O original é imutável e fica guardado**, e todo bug de parsing vira "corrigir e reprocessar".
3. **A unidade de trabalho é explícita**: o arquivo, ou um pedaço dele para arquivos gigantes.
4. **Workers não têm estado**: o status de cada arquivo mora no banco, e qualquer réplica pode assumir o trabalho de outra.
5. **O banco é protegido** com lotes, COPY, concorrência limitada e back-pressure.
6. **Todo passo é idempotente**, o que torna retries seguros.

A justificativa detalhada de cada um está no Módulo 2.1 do curso de .NET.

2.2 Os componentes, com as escolhas de Go

- **Ingestão:** a API em Go recebe o upload em corpo cru, calcula o SHA-256 com `io.TeeReader` enquanto envia ao S3 e insere o arquivo com `UNIQUE (cliente_id, sha256)`, gravando a mensagem na outbox na mesma transação (curso de APIs, Módulo 6.5).
- **Object storage:** S3 com chave previsível. O `GetObject` do SDK devolve um `io.ReadCloser` em streaming, e o parâmetro `Range` permite ler só um intervalo de bytes.
- **Fila de arquivos:** SQS com *long polling*. Cada mensagem carrega o ID do arquivo e a chave no storage, nunca o conteúdo.
- **Workers em Go:** o foco dos Módulos 3 a 7. Cada worker é um processo com algumas goroutines por arquivo: uma lê e monta lotes, algumas gravam, uma renova o lease. Canais com capacidade limitada ligam umas às outras.
- **DLQ:** a *redrive policy* da fila move mensagens que falharam N vezes.
- **Staging:** uma tabela temporária por transação, carregada com `CopyFrom` do pgx (Módulo 6).
- **Tabelas finais:** `INSERT ... SELECT ... ON CONFLICT DO NOTHING` com a chave natural (arquivo, linha), mais a outbox no mesmo comando.
- **Relay da outbox:** um processo Go pequeno com `FOR UPDATE SKIP LOCKED` (Módulo 8).

2.3 O ciclo de vida de um arquivo

Os estados (`RECEBIDO`, `PROCESSANDO`, `CONCLUIDO`, `REJEITADO`, `FALHOU`), o lease com `processando_ate` e a tabela `arquivo` são exatamente os do curso de .NET, Módulo 2.3, onde está o DDL completo. Mantenha um único dono das migrations: se os três sistemas (workers .NET, API Go, workers Go) usarem o mesmo banco, apenas um deles as aplica, e os outros as leem (no caso do Go, para o `sqlc generate`). Em Go, os estados viram um tipo string com constantes:

```
type StatusArquivo string

const (
    StatusRecebido    StatusArquivo = "RECEBIDO"
    StatusProcessando StatusArquivo = "PROCESSANDO"
    StatusConcluido   StatusArquivo = "CONCLUIDO"
    StatusRejeitado   StatusArquivo = "REJEITADO"
    StatusFalhou      StatusArquivo = "FALHOU"
)
```

Toda transição é um `UPDATE` condicional (`WHERE status = 'esperado'`), o que impede dois workers de pegarem o mesmo arquivo. A consulta de tomada com lease está no Módulo 8.

2.4 Arquivos gigantes: fan-out e fan-in

O padrão é o do curso de .NET (Módulo 2.4): calcular offsets de byte a cada N linhas (trivial em largura fixa: a linha k começa em $k \times 402$ bytes com CRLF), criar um registro por chunk, publicar uma mensagem por chunk e consolidar quando o último terminar, conferindo o trailer só no fan-in. Em Go, cada worker lê o intervalo do seu chunk com `GetObject` e `Range: aws.String("bytes=inicio-fim")`, e o resto do pipeline não muda: um `io.Reader` é um `io.Reader`, venha de um arquivo inteiro ou de um pedaço dele.

2.5 Alternativas

As mesmas escolhas e critérios do curso de .NET (Módulo 2.5): fila gerenciada ou tabela como fila com `SKIP LOCKED`, eventos do storage ou polling de pasta, staging com consolidação ou upsert direto, arquivo inteiro ou chunks. Uma observação específica de Go: como um worker é um binário pequeno que sobe em milissegundos, escalar de zero (Módulo 10) custa pouco, o que torna a fila gerenciada com autoscaling ainda mais atraente.

Exercício do Módulo 2

Desenhe a arquitetura para o seu cenário e, para cada seta, escreva o que acontece se o destino ficar fora do ar por 10 minutos: o que se perde, o que se acumula e como o sistema se recupera. Compare com a resposta que você deu (ou daria) no curso de .NET: ela deve ser idêntica, porque nada nesta análise depende da linguagem.

Módulo 3 — Runtime Go: o que o GC cobra e como não pagar

O objetivo é o mesmo do curso de .NET: **zero alocação por linha** no caminho quente. O motivo, em Go, é outro. O GC do Go não tem gerações: ele não tem uma gen0 barata onde objetos de vida curta morrem quase de graça. Cada ciclo marca todo o heap vivo, e quantos ciclos acontecem por segundo depende de quanto você aloca. Alocar menos é, literalmente, rodar o GC menos vezes. Este módulo explica a mecânica, os dois controles do runtime e as técnicas para escrever o parser sem alocar.

3.1 Como o GC do Go funciona

- **Concorrente:** a marcação roda em paralelo com o programa, usando parte da CPU (por padrão, cerca de 25% durante o ciclo), com pausas curtas no início e no fim.

- **Sem gerações e sem mover objetos:** não há gen0/gen1/gen2 nem compactação. Por isso também não há LOH nem o limite de 85.000 bytes do .NET; um slice grande é só um objeto grande.
- **Regulado pelo crescimento do heap:** com o padrão `GOGC=100`, um novo ciclo começa quando o heap cresce 100% sobre o heap vivo do ciclo anterior. Com 200 MB vivos, o próximo ciclo vem quando o heap chega a 400 MB.

Disso sai a conta que importa: **o custo de cada ciclo é proporcional ao heap vivo que ele precisa percorrer, e a frequência dos ciclos é proporcional à taxa de alocação.** Um parser que aloca 10 strings por linha a 35.000 linhas por segundo produz centenas de milhares de objetos por segundo e dispara ciclos sem parar.

E um detalhe que favorece Go neste problema: o GC **não varre o conteúdo de memória sem ponteiros**. Um `[]byte` de 100 MB ou um slice de structs só com números custa quase nada para marcar. Um slice de structs que contenham ponteiros (strings, `time.Time`, slices) precisa ser percorrido elemento por elemento. Por isso o registro do parser (3.5) é desenhado sem ponteiros.

O Green Tea, coletor padrão desde o Go 1.26, melhorou bastante a marcação de objetos pequenos (melhor localidade de memória e uso de instruções vetoriais em CPUs recentes). O princípio continua: menos alocação, menos trabalho.

3.2 GOGC e GOMEMLIMIT num worker

O runtime tem dois controles, e para um worker de lote vale combiná-los:

Controle	O que faz	Uso no worker
<code>GOGC</code>	Quanto o heap pode crescer entre ciclos (percentual sobre o vivo)	Subir para 200 a 400 troca memória por menos ciclos
<code>GOMEMLIMIT</code>	Limite "suave" de memória total do runtime; perto dele, o GC trabalha mais	Um pouco abaixo do limite do pod (por exemplo, 90%), para nunca levar OOM

Com os dois juntos, o GC roda pouco enquanto há folga (`GOGC` alto) e aperta automaticamente quando a memória se aproxima do teto (`GOMEMLIMIT`). Defina-os por variável de ambiente no deployment ou no código, na subida:

```
debug.SetGCPercent(300)           // equivalente a GOGC=300
debug.SetMemoryLimit(1800 << 20) // equivalente a GOMEMLIMIT=1800MiB,
para um pod de 2 GiB
```

Evite `GOGC=off` com `GOMEMLIMIT`: o GC só rodaria perto do limite, e se o heap vivo de fato chegar lá, o processo passa a gastar quase toda a CPU coletando. O `GOMAXPROCS` não precisa de ajuste: desde o Go 1.25 ele respeita o limite de CPU do container.

3.3 Pilha ou heap: escape analysis

Em Go você não escolhe onde um valor mora; o compilador decide. Valores que não "escapam" da função ficam na pilha e não custam nada ao GC. Para ver as decisões:

```
go build -gcflags=-m ./internal/cnab400
./parser.go:41:6: can inline digitos
./parser.go:58:23: linha does not escape
./worker.go:77:15: ... argument escapes to heap
```

O que costuma fazer um valor escapar, e portanto alocar:

- Guardar o valor numa interface (`any` , `error` , argumentos de `fmt.Sprintf` e de `slog`).
- Devolver um ponteiro para uma variável local, ou guardá-lo numa struct que vive mais.
- Capturar variáveis numa closure que é guardada ou executada em outra goroutine.
- Converter `[]byte` em `string` (copia os bytes), exceto em casos que o compilador otimiza, como comparação e busca em `map` .
- Criar slices cujo tamanho o compilador não conhece. O Go 1.26 passou a colocar mais desses slices na pilha, mas não conte com isso no caminho quente.

3.4 `[]byte` faz o papel do Span

Um slice em Go é um ponteiro, um tamanho e uma capacidade sobre um array existente. `linha[62:70]` não copia nada: é uma janela de 8 bytes, exatamente como `Slice(62, 8)` num `ReadOnlySpan<byte>` . O diagrama do `Span` no curso de .NET (Módulo 3.3) vale palavra por palavra para slices de Go.

A diferença perigosa: em C#, o compilador impede que um `Span` escape do método; em Go, nada impede você de guardar um slice. Isso importa porque o `bufio.Scanner` (Módulo 4) **reutiliza o mesmo buffer**: o slice devolvido por `Bytes()` é sobrescrito na próxima leitura. Guardar esse slice num registro é o bug mais comum de quem otimiza parsing em Go: os registros "mudam sozinhos". A regra é parsear na hora para valores (números, datas) e, se precisar guardar texto, copiar com `bytes.Clone` ou `string(b)` .

3.5 O parser sem alocação

```
package cnab400

const TamanhoLinha = 400

// Data no formato AAAAMMDD: um int32, sem ponteiros (time.Time tem um
// ponteiro para *Location)
type Data int32

func (d Data) Time() time.Time {
    return time.Date(int(d/10000), time.Month(d/100%100), int(d%100), 0, 0,
0, 0, time.UTC)
}

// Registro sem nenhum ponteiro: o GC não precisa varrer um slice deles
type Registro struct {
    NossoNumero      int64
    ValorPagoCentavos int64
    DataOcorrencia   Data
    NumeroLinha      int32
    CodigoOcorrencia uint8
}

// ParseDetalhe devolve o registro ou um motivo (string constante: não aloca)
func ParseDetalhe(linha []byte, numero int32) (Registro, string, bool) {
    if len(linha) != TamanhoLinha {
        return Registro{}, "tamanho inválido", false
    }
    if linha[0] != '1' {
        return Registro{}, "não é detalhe", false
    }
    // posições ilustrativas, 1-based e inclusivas no layout: 63 a 70 vira
[62:70]
    nosso, ok := digitos(linha[62:70])
    if !ok {
        return Registro{}, "nosso número", false
    }
    ocorrencia, ok := digitos(linha[108:110])
    if !ok {
        return Registro{}, "ocorrência", false
    }
    data, ok := dataDDMMAA(linha[110:116])
    if !ok {
        return Registro{}, "data", false
    }
}
```

```

    valor, ok := digitos(linha[253:266])
    if !ok {
        return Registro{}, "valor pago", false
    }
    return Registro{
        NossoNumero: nosso, ValorPagoCentavos: valor, DataOcorrencia: data,
        NumeroLinha: numero, CodigoOcorrencia: uint8(ocorrencia),
    }, "", true
}

// ParseTrailer lê quantidade de registros e valor total, usados na
reconciliação (Módulo 8)
func ParseTrailer(linha []byte) (quantidade int, totalCentavos int64, ok
bool) {
    if len(linha) != TamanhoLinha || linha[0] != '9' {
        return 0, 0, false
    }
    q, ok1 := digitos(linha[17:25])
    t, ok2 := digitos(linha[25:39])
    return int(q), t, ok1 && ok2
}

func digitos(b []byte) (int64, bool) {
    var v int64
    for _, c := range b {
        d := c - '0' // byte sem sinal: qualquer coisa fora de '0'..'9' vira
um valor maior que 9
        if d > 9 {
            return 0, false
        }
        v = v*10 + int64(d)
    }
    return v, true
}

func dataDDMMAA(b []byte) (Data, bool) {
    dd, ok1 := digitos(b[0:2])
    mm, ok2 := digitos(b[2:4])
    aa, ok3 := digitos(b[4:6])
    if !ok1 || !ok2 || !ok3 || mm < 1 || mm > 12 || dd < 1 {
        return 0, false
    }
    ano := 2000 + aa
    ultimoDia := time.Date(int(ano), time.Month(mm)+1, 0, 0, 0, 0, 0,
time.UTC).Day() // dia 0 do mês seguinte
    if dd > int64(ultimoDia) {

```

```

    return 0, false
}
return Data(ano*10000 + mm*100 + dd), true
}

```

O que observar, comparando com o parser em C#:

- **Registro sem ponteiros.** A data vira um `int32` em vez de `time.Time` (que guarda um ponteiro para o fuso). Um `[]Registro` de 5.000 itens é um bloco de 160 KB que o GC trata como um objeto opaco. A conversão para `time.Time` só acontece na hora de enviar ao banco, na pilha.
- **Ordem dos campos.** Os dois `int64` primeiro, depois os `int32`, depois o `uint8`. Go não reordena campos, e essa ordem minimiza o preenchimento por alinhamento: a struct ocupa 32 bytes. Intercalando pequenos e grandes (`uint8`, `int64`, `int32`, `int64`, `int32`), o preenchimento levaria a 40 bytes: 25% a mais de memória por lote.
- **Erro como string constante.** Literais de string não alocam. Devolver `fmt.Errorf(...)` aqui alocaria um erro por linha inválida.
- **Três retornos em vez de exceção.** `(Registro, string, bool)` é o equivalente ao `TryParse` com `out`: o caminho feliz não paga nada.
- **`time.Date` com `time.UTC` não aloca:** `time.Time` é um valor, e `time.UTC` é um ponteiro para uma variável global.

3.6 Reaproveitando memória

O equivalente ao `ArrayPool<T>` tem duas formas em Go:

- **`sync.Pool`**: um cache de objetos reaproveitáveis por goroutine, que o runtime esvazia aos poucos a cada GC. Bom para buffers temporários de tamanho variável (por exemplo, um `bytes.Buffer` para montar um evento JSON). Não limita quantos objetos existem.
- **Um canal de buffers livres:** um canal com capacidade N, pré-preenchido com N slices. Quem precisa de um buffer recebe do canal; quem termina devolve. Além de reaproveitar, ele **limita** a memória total a N buffers e, de quebra, cria back-pressure. É a técnica do pipeline do Módulo 5.

E o reaproveitamento mais simples de todos: `buf = buf[:0]` zera o tamanho de um slice mantendo a capacidade, como o `List<T>.Clear()`. O próximo `append` reutiliza a mesma memória.

Para campos de texto que precisam ser guardados (nome do pagador), arquivos bancários costumam vir em ISO-8859-1. Cada byte Latin-1 corresponde ao mesmo número de ponto de código Unicode, então a conversão é um laço curto com `utf8.AppendRune(dst,`

`rune(b))` sobre um buffer reaproveitado, ou o decodificador de `golang.org/x/text/encoding/charmap`. Faça isso só nos campos que realmente vão para o banco.

Exercício do Módulo 3

Escreva dois benchmarks com `testing.B` e `b.Loop()` (Go 1.24+; desde o 1.26 o corpo do laço pode ser inlined sem distorcer a medição) sobre 100 mil linhas: um com `strings.Split + strconv.ParseInt + time.Parse`, outro com o `ParseDetalhe` acima. Rode com `go test -bench . -benchmem` e compare ns/op e allocs/op: o segundo deve ter zero alocações. Depois troque `Data` por `time.Time` no Registro, rode `go build -gcflags=-m` e um perfil de heap, e observe o slice de registros passar a ser varrido pelo GC.

Módulo 4 — I/O de arquivos em streaming

O objetivo é ler um arquivo de qualquer tamanho com memória constante. Em Go isso é natural: tudo que lê bytes é um `io.Reader` (arquivo local, corpo de um `GetObject` do S3, um `gzip.Reader`, um pedaço de arquivo), e o `bufio.Scanner` transforma qualquer `io.Reader` numa sequência de linhas sem copiar nem alocar por linha. O papel que o `System.IO.Pipelines` cumpria no .NET, inclusive tratar a linha que cruza a borda do buffer, o `Scanner` cumpre sozinho.

4.1 Os níveis de leitura

Abordagem	Memória	Alocação por linha	Quando usar
<code>os.ReadFile + bytes.Split</code>	Proporcional ao arquivo	Um slice por linha, todos vivos	Arquivos pequenos de configuração
<code>bufio.Scanner + sc.Text()</code>	Constante	Uma string por linha	Volume moderado, protótipos
<code>bufio.Scanner + sc.Bytes()</code>	Constante	Zero	Alto volume: este curso
<code>bufio.Reader.ReadSlice('\n')</code>	Constante	Zero	Quando precisa de controle fino (linhas maiores que o buffer, por exemplo)

A diferença entre a segunda e a terceira linha é uma chamada: `Text()` copia a linha para uma string nova, `Bytes()` devolve uma janela sobre o buffer interno. A terceira exige a disciplina do Módulo 3.4: a janela é sobrescrita no próximo `Scan`.

4.2 O leitor do CNAB

O leitor lê, parseia e entrega cada registro válido a uma função `emitir`. Ele não sabe nada de lotes, canais ou banco; quem decide o que fazer com cada registro é o pipeline do Módulo 5. Essa separação deixa o leitor testável com um `strings.Reader` e uma função que guarda os registros num slice.

```
package leitura

// Contexto: o que o leitor informa sobre o arquivo, além dos registros
// válidos (implementado no Módulo 8)
type Contexto interface {
    RegistrarErro(linha int32, motivo string)
    RegistrarTrailer(quantidade int, totalCentavos int64)
    RegistrarFimDaLeitura(linhas int32)
}

var ErrLayoutInvalido = errors.New("layout de arquivo inválido")

func LerCnab400(origem io.Reader, c Contexto, emitir func(cnab400.Registro)
error) error {
    sc := bufio.NewScanner(origem)
    sc.Buffer(make([]byte, 256<<10), 256<<10) // buffer de 256 KiB; nenhuma
    linha pode passar disso

    var n int32
    for sc.Scan() {
        n++
        linha := sc.Bytes() // janela sobre o buffer: válida só até o próximo
        Scan
        if len(linha) == 0 {
            continue
        }
        switch linha[0] {
        case '0': // header: banco, tipo de arquivo, data de geração
            if !headerValido(linha) {
                return fmt.Errorf("linha %d: %w", n, ErrLayoutInvalido)
            }
        case '9': // trailer: totais para a reconciliação
            if q, total, ok := cnab400.ParseTrailer(linha); ok {
                c.RegistrarTrailer(q, total)
            }
        }
    }
}
```

```

        } else {
            c.RegistrarErro(n, "trailer inválido")
        }
    default:
        r, motivo, ok := cnab400.ParseDetalhe(linha, n)
        if !ok {
            c.RegistrarErro(n, motivo)
            continue
        }
        if err := emitir(r); err != nil { // aqui entra o back-pressure
(Módulo 5)
            return err
        }
    }
}
if err := sc.Err(); err != nil {
    return fmt.Errorf("leitura na linha %d: %w", n+1, err)
}
c.RegistrarFimDaLeitura(n)
return nil
}

```

O que o `Scanner` faz por você, e que no .NET exigia código:

- **Linhas que cruzam a borda do buffer:** o `Scanner` desloca o pedaço incompleto para o início do buffer e lê mais, sem cópia para buffers auxiliares. Não há `consumed / examined` para errar.
- **CRLF:** a função padrão de divisão (`bufio.ScanLines`) remove `\n` e também o `\r` que o precede. A linha chega com exatamente 400 bytes, venha o arquivo de Windows ou de Linux.
- **Última linha sem quebra:** é entregue normalmente.
- **Linha grande demais:** se uma linha passar do tamanho máximo do buffer, `Scan` devolve `false` e `sc.Err()` devolve `bufio.ErrTooLong`. Num arquivo CNAB, isso significa que não é um arquivo CNAB: o erro sobe e o arquivo é rejeitado.

4.3 Arquivos locais e leitura por intervalo

Para arquivos num disco local ou num volume montado (FSx via SMB, por exemplo), `os.Open` devolve um `*os.File`, que já é um `io.Reader`. O `Scanner` faz o buffering, então não é preciso um `bufio.Reader` no meio.

Para ler um chunk específico (o fan-out do Módulo 2.4), use `io.NewSectionReader(arquivo, inicio, tamanho)`: ele lê só aquele intervalo com `ReadAt`, que não depende da posição atual do arquivo e é seguro para várias goroutines

lerem chunks diferentes do mesmo `*os.File` ao mesmo tempo. É o equivalente da API `RandomAccess` do .NET.

4.4 Lendo direto do S3

O corpo de um `GetObject` é um `io.ReadCloser` em streaming. Passe-o direto para o leitor:

```
entrada := &s3.GetObjectInput{Bucket: aws.String(bucket), Key:
aws.String(chave)}
if chunk != nil {
    entrada.Range = aws.String(fmt.Sprintf("bytes=%d-%d", chunk.Inicio,
chunk.Fim)) // só o intervalo do chunk
}
saida, err := s3c.GetObject(ctx, entrada)
if err != nil {
    return fmt.Errorf("abrir %s: %w", chave, err)
}
defer saida.Body.Close()

err = leitura.LerCnab400(saida.Body, contexto, emitir)
```

Os dois cuidados do curso de .NET valem igual. Uma conexão pode cair no meio, e o erro chega como erro de leitura do `Scanner`: trate-o como transitório e reprocessse o arquivo inteiro, o que é seguro porque a escrita é idempotente. E se o pipeline ficar lento e o back-pressure parar a leitura por muito tempo, a conexão HTTP com o S3 pode expirar. Se isso acontecer com frequência, baixe primeiro para um arquivo temporário (`os.CreateTemp` + `io.Copy`, com `defer os.Remove`) e processe a partir do disco.

O `ctx` passado ao `GetObject` vale para a leitura do corpo também: se o contexto for cancelado (um erro em outro estágio, ou o desligamento), a próxima leitura do `Body` falha, e o `Scanner` para. Esse é o fio que conecta o cancelamento do Módulo 5 até a rede.

4.5 Hash enquanto lê

Para calcular o SHA-256 sem ler o arquivo duas vezes, ponha um `io.TeeReader` entre a origem e o destino: tudo que o destino lê passa também pelo hash.

```
h := sha256.New()
if _, err := io.Copy(destino, io.TeeReader(origem, h)); err != nil {
    return err
}
soma := hex.EncodeToString(h.Sum(nil))
```

O `io.Copy` usa um buffer de 32 KiB, e o SHA-256 da biblioteca padrão usa instruções específicas da CPU quando disponíveis; o hash raramente é o gargalo. Esse é o código da ingestão (curso de APIs, Módulo 6.5); o worker não precisa recalcular, mas pode fazê-lo durante a leitura para conferir a integridade do objeto contra o hash registrado.

4.6 Escrevendo arquivos posicionais

Para gerar arquivos de remessa ou relatórios em layout fixo, escreva bytes num buffer reaproveitado e envie em blocos por um `bufio.Writer`:

```
// acrescenta v com zeros à esquerda até 'largura' dígitos, sem alocar
func acrescentarNumero(dst []byte, v int64, largura int) []byte {
    var tmp [20]byte
    s := strconv.AppendInt(tmp[:0], v, 10) // tmp fica na pilha
    for range largura - len(s) {
        dst = append(dst, '0')
    }
    return append(dst, s...)
}

w := bufio.NewWriterSize(arquivo, 256<<10)
linha := make([]byte, 0, 402)
for _, r := range registros {
    linha = linha[:0]
    linha = append(linha, '1')
    // ... demais campos
    linha = acrescentarNumero(linha, r.ValorPagoCentavos, 13)
    linha = append(linha, '\r', '\n')
    if _, err := w.Write(linha); err != nil {
        return err
    }
}
if err := w.Flush(); err != nil { // sem o Flush, o último bloco nunca chega
    ao arquivo
    return err
}
```

Como no .NET, quem consome nunca deve ver um arquivo pela metade: grave num arquivo temporário no mesmo diretório e use `os.Rename` no fim (atômico no mesmo sistema de arquivos), ou envie ao S3 com o `manager.Uploader` do SDK, que só publica o objeto quando todas as partes chegam. Para enviar em streaming enquanto gera, ligue o gerador ao uploader com `io.Pipe`: o gerador escreve numa ponta, o uploader lê da outra, e nenhum dos dois precisa do arquivo inteiro na memória.

4.7 Arquivos compactados

`gzip.NewReader(origem)` funciona em streaming: é só mais um `io.Reader` na corrente. Arquivos `.zip` são diferentes, como no .NET: o índice fica no fim, então `zip.NewReader` exige um `io.ReaderAt` e o tamanho total. Com um objeto do S3, baixe para um arquivo temporário (que é um `io.ReaderAt`) e abra dali.

Exercício do Módulo 4

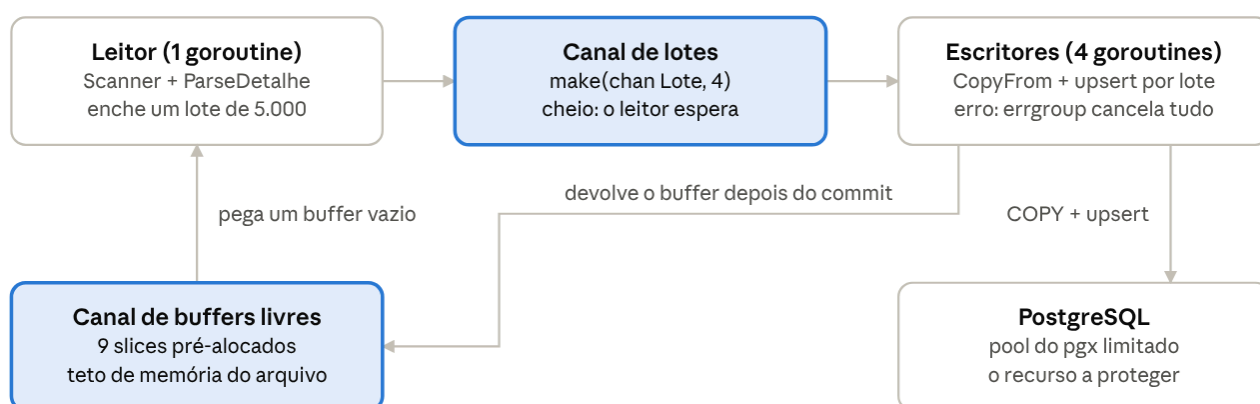
Gere um arquivo sintético de 2 milhões de linhas de 400 posições com CRLF (cerca de 800 MB). Leia-o com `sc.Text()` e depois com `LerCnab400`, usando um `emitir` que só conta registros. Compare tempo total, `allocs` do `runtime.MemStats` e número de ciclos de GC (`NumGC`). Depois reduza o buffer do `Scanner` para 1.000 bytes e confirme que as linhas que cruzam a borda continuam chegando inteiras; reduza para 300 bytes e veja o `bufio.ErrTooLong`.

Módulo 5 — Concorrência e back-pressure

Em Go, back-pressure não é uma configuração: é o comportamento padrão de um canal com buffer. Enviar para um canal cheio bloqueia a goroutine que envia até alguém receber, e receber de um canal vazio bloqueia quem recebe. O pipeline de um arquivo usa dois canais com limite: um leva lotes cheios do leitor aos escritores, o outro devolve os buffers vazios dos escritores ao leitor. Juntos, eles fixam a memória máxima e fazem a velocidade do banco chegar até a leitura do arquivo.

Dois canais com limite seguram a memória e fazem o banco ditar o ritmo

Pipeline de um arquivo dentro do worker · em destaque, os canais: cheio ou vazio, quem envia ou recebe espera



pipeline do worker · 3 estágios, 2 canais com limite

Compare com o diagrama do curso de .NET: lá havia um canal de registros e um agrupador separado, e os buffers voltavam para um `ArrayPool` sem limite. Aqui o leitor já agrupa (enviar um item por canal tem custo por operação, então se envia o lote inteiro), e o canal

de buffers livres faz o papel do pool e do teto de memória ao mesmo tempo.

5.1 Goroutines não são tarefas assíncronas

No .NET, concorrência de I/O era `async / await`, e bloquear uma thread do pool era um erro grave. Em Go o código é bloqueante e direto: `tx.CopyFrom(...)` simplesmente espera o banco. Enquanto espera, o runtime estaciona aquela goroutine e roda outra na mesma thread do sistema. Por isso não há ThreadPoo starvation, não há `.Result` perigoso e não há duas versões de cada função (síncrona e assíncrona).

O que continua valendo: trabalho de I/O pode ter muita concorrência (goroutines custam alguns KB), mas o trabalho de CPU só tem paralelismo até `GOMAXPROCS`. E concorrência sem limite contra o banco piora a vazão, exatamente como no .NET (Módulo 5.5 de lá).

5.2 Canais: as regras que evitam vazamentos

- **Quem envia fecha.** `close(ch)` sinaliza "não virão mais itens", e o `for item := range ch` do lado de quem recebe termina quando o canal fecha e esvazia. Só quem envia sabe quando acabou. Fechar um canal fechado, ou enviar para ele, é `panic`.
- **Todo envio e recebimento que pode bloquear tem uma saída por contexto.** Um `ch <- v` sozinho bloqueia para sempre se ninguém mais vai receber (porque o receptor falhou e saiu). Sempre:

```
select {
case lotes <- lote:
case <-ctx.Done():
    return ctx.Err() // alguém falhou: desista em vez de ficar preso
}
```

- **Canal com buffer é o `BoundedChannel` com `FullMode = Wait`.** Canal sem buffer (`make(chan T)`) é um encontro: quem envia espera quem recebe. Não existe canal ilimitado em Go, o que elimina por construção o erro de memória crescendo sem parar.

Goroutine presa num canal para sempre é o vazamento clássico de Go, e o perfil `goroutineleak` do Go 1.27 (Módulo 9) existe para encontrá-lo.

5.3 errgroup: o orquestrador

O pacote `golang.org/x/sync/errgroup` substitui o código de orquestração que o curso de .NET escreveu à mão (`Estagio`, `CancellationTokenSource` vinculado, `Task.WhenAll`):

```

g, ctx := errgroup.WithContext(ctx)
g.Go(func() error { /* estágio 1 */ })
g.Go(func() error { /* estágio 2 */ })
err := g.Wait() // espera todos; devolve o primeiro erro

```

Quando qualquer função devolve erro, o `ctx` derivado é cancelado, e todos os `select` com `<-ctx.Done()` dos outros estágios acordam. `Wait` espera todas as goroutines terminarem, então nenhuma fica órfã, e devolve o primeiro erro, que é a causa (os demais costumam ser só "contexto cancelado").

5.4 O pipeline de um arquivo

```

package pipeline

type Lote struct {
    ArquivoID uuid.UUID
    Numero    int
    Itens     []cnab400.Registro
}

type Escritor interface {
    Gravar(ctx context.Context, lote Lote) error
}

type Config struct {
    TamanhoLote    int // 5.000
    CapacidadeLotes int // 4
    Escritores     int // 4
}

type Pipeline struct {
    cfg      Config
    escritor Escritor
}

func (p *Pipeline) Processar(ctx context.Context, arquivoID uuid.UUID, origem
io.Reader, c leitura.Contexto) error {
    g, ctx := errgroup.WithContext(ctx)

    lotes := make(chan Lote, p.cfg.CapacidadeLotes)
    livres := make(chan []cnab400.Registro,
p.cfg.CapacidadeLotes+p.cfg.Escritores+1)
    for range cap(livres) {
        livres <- make([]cnab400.Registro, 0, p.cfg.TamanhoLote) // toda a
memória do arquivo, alocada uma vez

```

```

}

// Leitor: único que envia em 'lotes', portanto único que o fecha
g.Go(func() error {
    defer close(lotes)
    atual := <-livres
    numero := 0

    enviar := func(pegarOutro bool) error {
        numero++
        select {
            case lotes <- Lote{ArquivoID: arquivoID, Numero: numero, Itens:
atual}:
                case <-ctx.Done():
                    return ctx.Err()
                }
            if !pegarOutro {
                return nil
            }
            select {
                case atual = <-livres: // sem buffer livre, espera: segundo ponto
de back-pressure
                    return nil
                case <-ctx.Done():
                    return ctx.Err()
                }
            }
        }

    err := leitura.LerCnab400(origem, c, func(r cnab400.Registro) error {
        atual = append(atual, r) // cabe sempre: o lote é enviado antes
de passar da capacidade
        if len(atual) == cap(atual) {
            return enviar(true)
        }
        return nil
    })
    if err == nil && len(atual) > 0 {
        err = enviar(false) // último lote, incompleto
    }
    return err
})

// Escritores: recebem até o canal fechar e esvaziar
for range p.cfg.Escritores {
    g.Go(func() error {
        for lote := range lotes {

```

```

        err := p.escriptor.Gravar(ctx, lote)
        livres <- lote.Itens[:0] // nunca bloqueia: 'livres' tem
        lugar para todos os buffers
        if err != nil {
            return fmt.Errorf("lote %d: %w", lote.Numero, err)
        }
    }
    return nil
})
}

return g.Wait()
}

```

Percorra os cenários de falha, porque é neles que um pipeline concorrente costuma travar:

1. **O banco desacelera.** Os escritores demoram, o canal de lotes enche, o leitor bloqueia no envio, o `Scanner` para de ler e a conexão com o S3 para de puxar bytes. Memória constante.
2. **Um escritor falha.** Ele devolve o erro, o `errgroup` cancela o `ctx`, o leitor acorda no `select` e devolve `ctx.Err()`, o `defer close(lotes)` fecha o canal, os outros escritores terminam (os lotes restantes falham rápido com o contexto cancelado) e `Wait` devolve o erro original.
3. **O arquivo tem layout inválido.** O leitor devolve `ErrLayoutInvalido`, fecha o canal, os escritores gravam o que já estava na fila (inofensivo: é idempotente e o arquivo será marcado como rejeitado) e saem.
4. **O worker recebe SIGTERM.** O contexto pai é cancelado, todo mundo sai pelo mesmo caminho do item 2, e a mensagem não é confirmada (Módulo 8).

5.5 Por que o canal de buffers livres

O canal `livres` tem três funções de uma vez:

- **Reaproveitamento:** os 9 slices são alocados uma vez por arquivo e circulam. Nenhum lote novo é alocado durante o processamento.
- **Teto de memória exato:** 9 buffers × 5.000 registros × 32 bytes = cerca de 1,4 MB por arquivo, independentemente do tamanho do arquivo. Com 2 arquivos em paralelo por réplica, menos de 3 MB de lotes.
- **Posse explícita:** um buffer pertence a exatamente um dono por vez (no canal, com o leitor ou com um escritor). Quem o recebe pode usá-lo sem trava, e quem o envia não o toca mais. É o idioma de Go: compartilhe memória comunicando, em vez de comunicar compartilhando memória.

A capacidade `CapacidadeLotes + Escritores + 1` não é arbitrária: é o número máximo de buffers que podem existir ao mesmo tempo (4 no canal, 4 em gravação, 1 sendo preenchido). Por isso a devolução `livres <- ...` nunca bloqueia.

5.6 Vários arquivos ao mesmo tempo

O laço principal do worker (projeto final) recebe mensagens da fila e processa até 2 arquivos em paralelo, com um semáforo feito de canal:

```
sem := make(chan struct{}, 2)
for msg := range mensagens {
    select {
        case sem <- struct{}{}: // ocupa uma vaga (espera se as duas estão
            ocupadas)
        case <-ctx.Done():
            return
    }
    wg.Go(func() { // sync.WaitGroup.Go, Go 1.25
        defer func() { <-sem }()
        processarMensagem(ctx, msg)
    })
}
wg.Wait()
```

A conta de conexões é a do .NET: 2 arquivos × 4 escritores = 8 conexões por réplica, mais as de lease e controle. O `MaxConns` do pgxpool precisa comportar isso, e réplicas × `MaxConns` precisa caber no orçamento do banco.

5.7 Quando a ordem por chave importa

As duas opções do curso de .NET valem igual. Particionar por chave, com N canais e o leitor enviando cada registro para `canais[hash(chave)%N]`, um escritor por canal. Ou, quase sempre mais simples, gravar tudo em staging com o número da linha e ordenar na consolidação em SQL.

5.8 Armadilhas de concorrência em Go

- **Envio ou recebimento sem `select` com contexto:** a goroutine vaza quando o outro lado sai.
- **Fechar o canal do lado de quem recebe**, ou de mais de um lugar: `panic` em produção.
- **Acessar o mesmo slice ou mapa de duas goroutines** sem sincronização: corrida de dados, com resultados imprevisíveis. Rode sempre `go test -race`.

- **Goroutine sem dono:** toda goroutine iniciada precisa de alguém que espere por ela (`errgroup` , `WaitGroup`) e de um caminho para terminar.
- **Guardar `sc.Bytes()`** (Módulo 3.4) num registro que vai para outro estágio: o buffer muda antes de o escritor ler.

Exercício do Módulo 5

Monte o `Pipeline` com um `Escritor` falso que faz `time.Sleep(50 * time.Millisecond)` por lote. Rode o arquivo do Módulo 4 e confirme, pelo `runtime.MemStats` , que a memória fica estável e que não há alocação de lotes durante o processamento. Depois faça o escritor falhar no lote 10 e confirme que `Processar` devolve esse erro e que nenhuma goroutine ficou viva (`runtime.NumGoroutine()` antes e depois). Por fim, remova o `case <-ctx.Done()` do envio do leitor, repita a falha e observe o programa travar: é o vazamento que o `select` evita.

Módulo 6 — Escrita em massa no banco

A estratégia é exatamente a do curso de .NET, porque ela é do PostgreSQL e não da linguagem: para cada lote, uma transação curta que carrega o lote numa tabela temporária com `COPY` e consolida na tabela final com um único `INSERT ... SELECT ... ON CONFLICT DO NOTHING` , gravando a outbox no mesmo comando. Em Go, o `COPY` é o `CopyFrom` do `pgx`, e o resto é SQL.

6.1 As técnicas, em Go

Técnica	Idas ao banco por lote de 5.000	Prós	Contras
<code>tx.Exec</code> por linha	5.000	Simples	Inaceitável em volume
<code>pgx.Batch</code> (comandos em pipeline)	1	Envia vários comandos de uma vez	Ainda 5.000 <code>INSERT</code> para o servidor planejar e executar
<code>INSERT ... SELECT FROM unnest(\$1::bigint[], ...)</code>	1	Um parâmetro por coluna; funciona com <code>PgBouncer</code>	Montar um slice por coluna

Técnica	Idas ao banco por lote de 5.000	Prós	Contras
<code>tx.CopyFrom</code>	1 (stream)	O mais rápido; protocolo feito para carga	Não faz upsert sozinho: precisa de staging

Como no .NET, a combinação vencedora é a última linha mais uma consolidação em SQL. O motivo de cada custo (ida e volta de rede, planejamento, commit, índices) está no Módulo 6.1 do curso de .NET.

6.2 O modelo de dados

As tabelas `retorno_detalhe` e `outbox`, com a chave natural `UNIQUE (arquivo_id, numero_linha)` que torna a escrita idempotente e o índice parcial da `outbox` pendente, são as do Módulo 6.2 do curso de .NET, onde o DDL está comentado coluna a coluna. Os valores em centavos são `bigint`, coerentes com o `int64` do `cnab400.Registro`.

As consultas de controle (tomar o arquivo, renovar o lease, reconciliar) ficam no `sqlc`, como no curso de APIs. A carga em massa usa o `pgx` diretamente, porque é onde o controle fino compensa.

6.3 O escritor

```
package escrita

const criarTemporaria = `
CREATE TEMP TABLE IF NOT EXISTS tmp_retorno (
    arquivo_id uuid, numero_linha int, nosso_numero bigint,
    codigo_ocorrencia smallint, data_ocorrencia date, valor_pago_centavos
    bigint
) ON COMMIT DELETE ROWS`

const consolidar = `
WITH inseridos AS (
    INSERT INTO retorno_detalhe AS d
        (arquivo_id, numero_linha, nosso_numero, codigo_ocorrencia,
        data_ocorrencia, valor_pago_centavos)
    SELECT arquivo_id, numero_linha, nosso_numero, codigo_ocorrencia,
        data_ocorrencia, valor_pago_centavos
    FROM tmp_retorno
    ON CONFLICT (arquivo_id, numero_linha) DO NOTHING
    RETURNING d.id, d.nosso_numero, d.codigo_ocorrencia,
        d.valor_pago_centavos
```

```

)
INSERT INTO outbox (id, tipo, chave, payload)
SELECT gen_random_uuid(), 'retorno.ocorrencia', i.nosso_numero::text,
       jsonb_build_object('detalheId', i.id, 'nossoNumero', i.nosso_numero,
                          'ocorrencia', i.codigo_ocorrencia,
                          'valorPagoCentavos', i.valor_pago_centavos)
FROM inseridos i`

var colunas = []string{"arquivo_id", "numero_linha", "nosso_numero",
                      "codigo_ocorrencia", "data_ocorrencia", "valor_pago_centavos"}

type Postgres struct {
    pool *pgxpool.Pool
}

func (e *Postgres) Gravar(ctx context.Context, lote pipeline.Lote) error {
    arquivo := [16]byte(lote.ArquivoID) // uuid.UUID é um [16]byte; a
    conversão explícita evita depender de reflexão no driver

    return pgx.BeginFunc(ctx, e.pool, func(tx pgx.Tx) error {
        if _, err := tx.Exec(ctx, criarTemporaria); err != nil {
            return fmt.Errorf("staging: %w", err)
        }

        linhas := pgx.CopyFromSlice(len(lote.Itens), func(i int) ([]any,
error) {
            r := &lote.Itens[i]
            return []any{arquivo, r.NumeroLinha, r.NossoNumero,
int16(r.CodigoOcorrencia),
                r.DataOcorrencia.Time(), r.ValorPagoCentavos}, nil
        })
        if _, err := tx.CopyFrom(ctx, pgx.Identifier{"tmp_retorno"}, colunas,
linhas); err != nil {
            return fmt.Errorf("copy: %w", err)
        }

        if _, err := tx.Exec(ctx, consolidar); err != nil {
            return fmt.Errorf("consolidar: %w", err)
        }
        return nil // commit; qualquer erro acima faz rollback
    })
}

```

Ponto a ponto, comparando com o escritor em C#:

- **pgx.BeginFunc** é o `await using var tx` mais o `CommitAsync`: commit se a função devolve `nil`, rollback se devolve erro ou entra em pânico. Não há como esquecer o rollback.
- **Tabela temporária criada uma vez por conexão.** `IF NOT EXISTS ... ON COMMIT DELETE ROWS` é o refinamento descrito no curso de .NET: como o `pgxpool` reaproveita conexões, a tabela sobrevive entre lotes e é esvaziada a cada commit, sem escrever nos catálogos do sistema a cada lote.
- **Idempotência automática.** `ON CONFLICT DO NOTHING` ignora linhas já gravadas, e o `RETURNING` só devolve as inseridas agora, então regravar um lote não gera eventos duplicados na outbox. O diagrama de duas tentativas do curso de .NET (Módulo 8.2) mostra o efeito.
- **Uma consolidação set-based:** um plano para 5.000 linhas, usando o índice único para detectar conflitos.

Um custo que existe em Go e não existia em C#: o `CopyFromSlice` pede um `[]any` por linha, e colocar `int64` e `time.Time` dentro de `any` aloca. São objetos de vida curtíssima e o banco continua sendo o gargalo, então raramente importa; meça antes de otimizar. Se o perfil mostrar o contrário, implemente a interface `pgx.CopyFromSource` numa struct que percorre o lote, e confira no perfil de memória se ganhou algo.

6.4 Alternativas

unnest, quando `COPY` não é viável (PgBouncer em modo transação com restrições, por exemplo). O `pgx` codifica slices de Go como arrays do PostgreSQL diretamente:

```
nossos := make([]int64, len(lote.Itens))
linhas := make([]int32, len(lote.Itens))
// ... um slice por coluna, preenchidos num laço
_, err := tx.Exec(ctx, `
    INSERT INTO retorno_detalhe (arquivo_id, numero_linha, nosso_numero,
    codigo_ocorrencia, data_ocorrencia, valor_pago_centavos)
    SELECT $1, * FROM unnest($2::int[], $3::bigint[], $4::smallint[],
    $5::date[], $6::bigint[])
    ON CONFLICT (arquivo_id, numero_linha) DO NOTHING`,
    arquivo, linhas, nossos, ocorrencias, datas, valores)
```

sqlc :copyfrom: o `sqlc` gera uma função tipada de `CopyFrom` a partir de uma anotação na consulta, recebendo um slice de structs de parâmetros. É mais seguro contra erro de ordem de colunas, ao custo de converter o lote para outro tipo de struct antes de enviar.

6.5 ON CONFLICT, MERGE e os ajustes do banco

Tudo do curso de .NET (Módulos 6.5 e 6.8) se aplica sem mudança, porque é PostgreSQL: `ON CONFLICT DO NOTHING` para fatos imutáveis e `DO UPDATE ... WHERE` para estado; a restrição de não tocar a mesma linha duas vezes num comando (deduplique com `DISTINCT ON`); `MERGE` só quando as chaves de transações concorrentes não se sobrepõem; lotes de 1.000 a 10.000 linhas; transações curtas para não segurar o autovacuum; `synchronous_commit = off` só com reconciliação e dado reconstruível; ajuste do autovacuum para tabelas só de inserção.

Um ajuste específico do pgx: o `MaxConns` do pool precisa ser pelo menos (arquivos em paralelo × escritores) + as conexões de controle. Se for menor, os escritores ficam esperando conexão dentro do `BeginFunc`, e o sintoma no perfil é tempo gasto em `pgxpool.(*Pool).Acquire`.

Exercício do Módulo 6

Num PostgreSQL local com as tabelas do curso de .NET, implemente três escritores (um `Exec` por linha, `unnest` e `CopyFrom` com consolidação) e grave 1 milhão de registros com cada um, em lotes de 5.000 e 4 escritores. Compare linhas por segundo. Depois rode o `CopyFrom` de novo com o mesmo arquivo e confirme que a contagem de `retorno_detalhe` e a da `outbox` não mudaram. Se tiver feito o exercício equivalente em .NET, compare os números: eles devem ficar próximos, porque o trabalho pesado acontece no banco.

Módulo 7 — Leitura em massa

As regras do curso de .NET valem integralmente: só as colunas necessárias, linhas consumidas em streaming, keyset em vez de `OFFSET`, índices que transformam a consulta numa varredura curta e particionamento quando a tabela passa de centenas de milhões de linhas. Em Go, o streaming é o comportamento padrão do pgx, e a exportação em massa usa o `COPY TO` direto para um `io.Writer`.

7.1 Streaming de linhas é o padrão

O `pool.Query` do pgx devolve um `pgx.Rows` que lê as linhas da rede conforme você chama `Next()`. Nada é materializado até você pedir:

```

linhas, err := pool.Query(ctx,
    `SELECT id, nosso_numero, valor_pago_centavos
    FROM retorno_detalhe
    WHERE data_ocorrencia = $1`, dia)
if err != nil {
    return err
}
defer linhas.Close()

var (
    id, nosso, valor int64
)
for linhas.Next() {
    if err := linhas.Scan(&id, &nosso, &valor); err != nil {
        return err
    }
    // processa uma linha por vez; memória constante
}
return linhas.Err() // erros de rede no meio aparecem aqui, não no Query

```

Três cuidados. `defer linhas.Close()` devolve a conexão ao pool mesmo se o laço sair cedo. `linhas.Err()` depois do laço é obrigatório: uma conexão que cai na linha 300 mil termina o laço normalmente, e só o `Err` conta que faltou o resto. E `pgx.CollectRows` (que coleta tudo num slice) é ótimo para resultados pequenos e errado para grandes, pelo mesmo motivo do `ToListAsync` no .NET.

O mesmo aviso sobre snapshots longos do curso de .NET vale aqui: enquanto o laço roda, a consulta segura um snapshot do MVCC. Jobs longos devem ler em páginas curtas (7.2), não numa consulta de 40 minutos.

7.2 Keyset em vez de OFFSET

O motivo (o `OFFSET` lê e descarta tudo antes da página) está no Módulo 7.2 do curso de .NET, com o diagrama dos dois custos. A versão em Go de um job que percorre um dia inteiro em páginas de 5.000:

```

var ultimoID int64
for {
    n := 0
    linhas, err := pool.Query(ctx,
        `SELECT id, nosso_numero, valor_pago_centavos
        FROM retorno_detalhe
        WHERE data_ocorrencia = $1 AND id > $2
        ORDER BY id
        LIMIT 5000`, dia, ultimoID)

```

```

    if err != nil {
        return err
    }
    for linhas.Next() {
        var d Detalhe
        if err := linhas.Scan(&d.ID, &d.NossoNumero, &d.ValorPagoCentavos);
err != nil {
            linhas.Close()
            return err
        }
        processar(d)
        ultimoID = d.ID
        n++
    }
    linhas.Close()
    if err := linhas.Err(); err != nil {
        return err
    }
    if n < 5000 {
        return nil // última página
    }
}

```

Com o índice `(data_ocorrencia, id)`, cada página custa o mesmo, e cada consulta é curta. Para ordenação por mais de uma coluna, a comparação de tuplas do PostgreSQL funciona igual: `WHERE (data_ocorrencia, id) > ($1, $2)`. E para paralelizar um reprocessamento, divida o intervalo de ids em faixas e dê uma faixa para cada goroutine de um `errgroup` com `SetLimit`.

7.3 Exportações: COPY TO direto para um io.Writer

Para exportar milhões de linhas, o `pgx` expõe o `COPY TO` na conexão de baixo nível, escrevendo direto num `io.Writer` (um arquivo, um `bufio.Writer`, a ponta de escrita de um `io.Pipe` ligado a um upload para o S3):

```

conn, err := pool.Acquire(ctx)
if err != nil {
    return err
}
defer conn.Release()

_, err = conn.Conn().PgConn().CopyTo(ctx, destino,
    `COPY (SELECT nosso_numero, valor_pago_centavos
        FROM retorno_detalhe WHERE data_ocorrencia = '2026-10-05')
    TO STDOUT WITH (FORMAT csv, HEADER true, DELIMITER ';')`)

```

O banco gera o CSV e o Go só move bytes: nenhuma linha passa por `Scan`, nenhum valor é convertido. É a exportação mais rápida possível. Como no .NET, `COPY` não aceita parâmetros: valores variáveis devem ser formatados com tipos seguros (uma `time.Time` formatada com `time.DateOnly`, nunca texto vindo de fora), ou encapsulados numa função SQL.

7.4 sqlc, pgx puro ou database/sql

Ferramenta	Melhor para	Observações
sqlc	Consultas de controle e de negócio com parâmetros e resultado tipados	SQL validado contra o schema na geração; o equivalente mais próximo de um ORM leve
pgx puro (<code>Query</code> , <code>CopyFrom</code> , <code>CopyTo</code>)	Caminho quente, carga, exportação, recursos específicos do PostgreSQL	Controle total, mais código
database/sql com pgx como driver	Bibliotecas que exigem a interface padrão	Perde <code>CopyFrom</code> e parte dos tipos nativos

No worker deste curso, sqlc e pgx puro convivem: sqlc para o ciclo de vida do arquivo, pgx para lotes e exportações.

7.5 Índices, particionamento e réplicas

Nada disso muda com a linguagem; os Módulos 7.5 a 7.7 do curso de .NET valem palavra por palavra: colunas de igualdade antes das de intervalo, `INCLUDE` para *index-only scans*, índices parciais, BRIN para colunas que crescem com a ordem física, `EXPLAIN (ANALYZE, BUFFERS)` para ler o plano real, particionamento por mês com a coluna de partição em toda chave única (o que muda a chave natural de idempotência para arquivo, linha e data) e réplicas de leitura para relatórios, com o cuidado do atraso de replicação.

Um detalhe prático do lado do Go: o pgx prepara e guarda em cache as consultas automaticamente por conexão. Isso acelera consultas repetidas, mas consome memória no servidor por conexão e conflita com PgBouncer em modo transação. Nesse cenário, configure o modo de execução de consultas do pgx para não usar statements preparados nomeados (a opção `DefaultQueryExecMode` da configuração da conexão).

Exercício do Módulo 7

Com alguns milhões de linhas em `retorno_detalle`, escreva o job de keyset acima e meça o tempo da página 1 e da página 500; repita com `OFFSET` e compare com `EXPLAIN (ANALYZE, BUFFERS)`. Depois exporte um dia inteiro com `CopyTo` para um arquivo e

compare o tempo com a mesma exportação feita linha a linha com `Scan` e `encoding/csv`. Por fim, derrube a conexão no meio de um laço de `Next()` e confirme que só o `linhas.Err()` revela a falha.

Módulo 8 — Confiabilidade: quando algo quebra no meio

Um pipeline confiável é aquele em que qualquer passo pode ser repetido com segurança. O mapa de falhas, a idempotência camada por camada e o diagrama de duas tentativas do curso de .NET (Módulos 8.1 e 8.2) valem sem mudança. Este módulo implementa cada peça em Go: o lease com renovação numa goroutine, o retry com backoff e jitter (sem biblioteca), as regras de confirmação da fila, o relay da outbox e a reconciliação com o trailer.

8.1 Tomando o arquivo com lease

O `UPDATE` atômico é o mesmo do curso de .NET, agora como consulta do sqlc:

```
-- name: TentarAssumir :one
UPDATE arquivo
SET status = 'PROCESSANDO',
    processando_ate = now() + interval '5 minutes',
    worker_id = @worker_id,
    tentativas = tentativas + 1
WHERE id = @id
    AND (status = 'RECEBIDO' OR (status = 'PROCESSANDO' AND processando_ate <
now()))
RETURNING tentativas;

-- name: RenovarLease :execrows
UPDATE arquivo SET processando_ate = now() + interval '5 minutes'
WHERE id = @id AND worker_id = @worker_id AND status = 'PROCESSANDO';
```

`TentarAssumir` devolvendo `pgx.ErrNoRows` significa que outro worker está com o arquivo (ou ele já terminou). `RenovarLease` com zero linhas afetadas significa que o lease foi perdido. O `worker_id` é o nome do pod (`os.Hostname()` no Kubernetes).

A renovação roda numa goroutine ao lado do pipeline. O detalhe que importa é o ciclo de vida: ela precisa parar quando o pipeline termina, e precisa cancelar o pipeline se o lease for perdido.

```
func manterLease(ctx context.Context, q *banco.Queries, fila Fila, msg
Mensagem, worker string) error {
    t := time.NewTicker(time.Minute)
    defer t.Stop()
```

```

    for {
        select {
            case <-ctx.Done():
                return nil // o processamento terminou (ou foi cancelado): fim
        normal
            case <-t.C:
                n, err := q.RenovarLease(ctx, banco.RenovarLeaseParams{ID:
msg.ArquivoID, WorkerID: worker})
                if err != nil {
                    return fmt.Errorf("renovar lease: %w", err)
                }
                if n == 0 {
                    return ErrLeasePerdido // outro worker assumiu: devolver erro
cancela o pipeline
                }
                if err := fila.EstenderVisibilidade(ctx, msg, 5*time.Minute); err
!= nil {
                    return fmt.Errorf("estender visibilidade: %w", err)
                }
            }
        }
    }
}

```

No projeto final, `manterLease` e o pipeline rodam no mesmo `errgroup`, com um contexto de lease que o pipeline cancela ao terminar. Se o lease falhar, o `errgroup` cancela o pipeline; se o pipeline terminar, ele cancela o lease. Os dois sempre param juntos.

8.2 Retry com backoff exponencial e jitter

Go não tem um Polly na biblioteca padrão, e uma função de vinte linhas cobre o que o worker precisa:

```

func ComRetry(ctx context.Context, tentativas int, base time.Duration, fn
func(context.Context) error) error {
    var err error
    for i := range tentativas {
        if err = fn(ctx); err == nil || !transitorio(err) {
            return err // sucesso, ou erro permanente: não adianta repetir
        }
        if i == tentativas-1 {
            break // última tentativa: não espera à toa
        }
        espera := rand.N(base << i) // "full jitter": aleatório entre 0 e
base * 2^i
        t := time.NewTimer(espera)
        select {

```

```

        case <-t.C:
        case <-ctx.Done():
            t.Stop()
            return errors.Join(err, ctx.Err())
        }
    }
    return err
}

func transitorio(err error) bool {
    if pgErr, ok := errors.AsType[*pgconn.PgError](err); ok {
        switch {
            case pgErr.Code == "40001", pgErr.Code == "40P01": // falha de
serialização, deadlock
                return true
            case pgErr.Code == "57P01": // servidor sendo desligado (failover)
                return true
            case strings.HasPrefix(pgErr.Code, "08"): // classe de erros de
conexão
                return true
        }
        return false // violação de constraint, sintaxe etc.: permanente
    }
    return pgconn.SafeToRetry(err) || errors.Is(err, io.ErrUnexpectedEOF)
}

```

O `rand.N` de `math/rand/v2` é genérico e aceita `time.Duration` direto. O *full jitter* (um valor aleatório entre zero e o teto exponencial) espalha as novas tentativas de muitos workers que falharam ao mesmo tempo, que é exatamente o problema que o jitter resolve no curso de .NET. `pgconn.SafeToRetry` diz se o erro aconteceu antes de a requisição chegar ao servidor (portanto, sem risco de ter sido executada).

Como no .NET, o retry entra como **decorator** do escritor, sem mexer no pipeline:

```

type EscriitorComRetry struct{ Interno pipeline.Escriitor }

func (e EscriitorComRetry) Gravar(ctx context.Context, l pipeline.Lote) error {
    return ComRetry(ctx, 5, 200*time.Millisecond, func(ctx context.Context)
error {
        return e.Interno.Gravar(ctx, l) // seguro de repetir: a transação
inteira é idempotente
    })
}

```

8.3 DLQ e as regras de confirmação

As regras são as do curso de .NET (Módulo 8.5) e merecem ser repetidas, porque erros aqui perdem dados:

- **Confirme (apague) a mensagem só depois do commit final** do arquivo, ou seja, depois da reconciliação e da mudança de status.
- **Erro permanente** (layout inválido): marque **REJEITADO** e confirme. Repetir não muda o conteúdo.
- **Erro transitório que sobreviveu aos retries**: não confirme. A fila reentrega depois do visibility timeout, e a *redrive policy* move a mensagem para a DLQ depois de N recebimentos.
- **Tentativas esgotadas** (o contador **tentativas** da tabela acima do limite): marque **FALHOU** e confirme, como segunda barreira independente da fila.
- **SIGTERM**: não confirme. O lease expira, outra réplica assume, e a idempotência cuida do que já foi gravado.

8.4 O relay da outbox

```
func (r *Relay) publicarLote(ctx context.Context) (int, error) {
    var publicados int
    err := pgx.BeginFunc(ctx, r.pool, func(tx pgx.Tx) error {
        linhas, err := tx.Query(ctx, `
            SELECT id::text, tipo, chave, payload::text
            FROM outbox
            WHERE publicado_em IS NULL
            ORDER BY criado_em
            LIMIT 200
            FOR UPDATE SKIP LOCKED`)
        if err != nil {
            return err
        }
        eventos, err := pgx.CollectRows(linhas, pgx.RowToStructByPos[Evento])
        // no máximo 200: coletar é ok
        if err != nil || len(eventos) == 0 {
            return err
        }
        if err := r.publicador.Publicar(ctx, eventos); err != nil { // em
lote, com o id como ID da mensagem
            return err
        }
        ids := make([]string, len(eventos))
        for i, e := range eventos {
            ids[i] = e.ID
        }
    })
    return publicados, err
}
```

```

    }
    if _, err := tx.Exec(ctx, `UPDATE outbox SET publicado_em = now()
WHERE id = ANY($1::uuid[])`, ids); err != nil {
        return err
    }
    publicados = len(eventos)
    return nil
})
return publicados, err
}

```

O laço externo chama `publicarLote` repetidamente e dorme meio segundo quando não há nada para publicar, sempre com `select` no contexto para sair no desligamento. As propriedades são as do curso de .NET: `SKIP LOCKED` permite várias réplicas do relay, a publicação é pelo menos uma vez (consumidores deduplicam pelo ID do evento), e a ordem global não é garantida entre réplicas.

8.5 Reconciliação com o trailer

O `Contexto` do leitor (Módulo 4.2) ganha sua implementação:

```

type ContextoDoArquivo struct {
    LinhasLidas          int32
    LinhasRejeitadas     int
    QuantidadeNoTrailer  int
    TotalNoTrailer       int64
    TemTrailer           bool
    PrimeirosErros       []ErroDeLinha
    limiteDeErros        int
}

type ErroDeLinha struct {
    Linha  int32
    Motivo string
}

func (c *ContextoDoArquivo) RegistrarErro(linha int32, motivo string) {
    c.LinhasRejeitadas++
    if len(c.PrimeirosErros) < c.limiteDeErros {
        c.PrimeirosErros = append(c.PrimeirosErros, ErroDeLinha{linha,
motivo})
    }
}

func (c *ContextoDoArquivo) RegistrarTrailer(q int, total int64) {
    c.QuantidadeNoTrailer, c.TotalNoTrailer, c.TemTrailer = q, total, true
}

```

```

}

func (c *ContextoDoArquivo) RegistrarFimDaLeitura(n int32) { c.LinhasLidas =
n }

```

Ele não precisa de mutex porque só a goroutine do leitor escreve nele, e só é lido depois de `g.Wait()`, que garante que tudo que o leitor escreveu está visível. A regra de reconciliação é a do curso de .NET (Módulo 8.7): sem trailer, `FALHOU`; registros gravados mais rejeitados devem bater com o trailer (ajustando se o layout do banco conta header e trailer); o total em valor só é comparado quando não há rejeições. A consulta de contagem e soma, a comparação e a mudança de status acontecem numa única transação.

8.6 Desligamento gracioso do worker

O `signal.NotifyContext` do `main` cancela o contexto raiz no `SIGTERM`. A partir daí, tudo desce em cascata pelo mesmo caminho dos erros: o laço da fila para de receber, cada pipeline em andamento sai pelos `select` com `<-ctx.Done()`, o lease para, e nenhuma mensagem incompleta é confirmada. O `main` espera o `WaitGroup` dos arquivos em andamento, com um prazo menor que o `terminationGracePeriodSeconds` do pod. Opcionalmente, antes de sair, libere os leases explicitamente (`status = 'RECEBIDO'`), para que outra réplica assuma na hora em vez de esperar a expiração.

Exercício do Módulo 8

O mesmo teste de caos do curso de .NET: processe um arquivo de 2 milhões de linhas com 2 réplicas e mate uma com `kill -9` no meio, três vezes em momentos diferentes. Verifique que a contagem de `retorno_detalhe` bate com o trailer, que não há dois eventos na outbox para a mesma linha e que o arquivo terminou `CONCLUIDO`. Depois derrube o PostgreSQL por 30 segundos durante o processamento e confirme que o `ComRetry` absorve a queda. Escreva também um teste de tabela para `transitorio`, com um `*pgconn.PgError` para cada código relevante.

Módulo 9 — Observabilidade e medição de performance

As mesmas quatro perguntas do curso de .NET: vazão (linhas por segundo), atraso (idade do arquivo mais antigo na fila), erros (rejeições, retries, DLQ) e saturação (canais cheios, pool de conexões, GC). A tabela de métricas do Módulo 9.1 de lá é a especificação; aqui ela é implementada com o OpenTelemetry para Go, e Go acrescenta duas ferramentas que valem ouro num worker: o `pprof` embutido e o perfil de goroutines vazadas.

9.1 Métricas com OpenTelemetry

```

type Metricas struct {
    linhasGravadas      metric.Int64Counter
    arquivosProcessados metric.Int64Counter
    duracaoArquivo      metric.Float64Histogram
    duracaoLote         metric.Float64Histogram
}

func NovasMetricas() (*Metricas, error) {
    m := otel.Meter("cobranca.retorno")
    var (
        r   Metricas
        err error
    )
    if r.linhasGravadas, err = m.Int64Counter("retorno.linhas.gravadas",
metric.WithUnit("{linha}")); err != nil {
        return nil, err
    }
    if r.arquivosProcessados, err =
m.Int64Counter("retorno.arquivos.processados", metric.WithUnit("{arquivo}"));
err != nil {
        return nil, err
    }
    if r.duracaoArquivo, err = m.Float64Histogram("retorno.arquivo.duracao",
metric.WithUnit("s")); err != nil {
        return nil, err
    }
    if r.duracaoLote, err = m.Float64Histogram("retorno.lote.duracao",
metric.WithUnit("ms")); err != nil {
        return nil, err
    }
    return &r, nil
}

func (m *Metricas) LoteGravado(ctx context.Context, linhas int, d
time.Duration) {
    m.linhasGravadas.Add(ctx, int64(linhas))
    m.duracaoLote.Record(ctx, float64(d.Microseconds())/1000)
}

func (m *Metricas) ArquivoFinalizado(ctx context.Context, status, banco
string, d time.Duration) {
    atributos := metric.WithAttributes(attribute.String("status", status),

```

```
attribute.String("banco", banco))
    m.arquivosProcessados.Add(ctx, 1, atributos)
    m.duracaoArquivo.Record(ctx, d.Seconds(), atributos)
}
```

A regra de cardinalidade é a mesma: `status` e `banco` como atributos, nunca `arquivo_id`. E a métrica de diagnóstico mais útil continua sendo a **ocupação dos canais**. Em Go, `len(ch)` é seguro de ler de qualquer goroutine, então um gauge observável lê a ocupação no momento da coleta, sem custo no caminho quente: canal de lotes sempre cheio indica gargalo no banco; canal de buffers livres sempre vazio indica o mesmo do outro lado; canal de lotes sempre vazio indica gargalo na leitura.

9.2 Traces e o contexto atravessando a fila

Um span por arquivo, aberto quando a mensagem é recebida, e spans filhos para leitura e para cada lote (com amostragem: não um por linha). A parte interessante é continuar o trace iniciado na API: a ingestão grava o `traceparent` nos atributos da mensagem, e o worker o extrai com o propagador do OpenTelemetry e um adaptador que implementa `propagation.TextMapCarrier` sobre os atributos da mensagem do SQS:

```
type atributosSQS map[string]types.MessageAttributeValue

func (a atributosSQS) Get(k string) string {
    if v, ok := a[k]; ok && v.StringValue != nil {
        return *v.StringValue
    }
    return ""
}

func (a atributosSQS) Set(k, v string) {
    a[k] = types.MessageAttributeValue{DataType: aws.String("String"),
    StringValue: aws.String(v)}
}

func (a atributosSQS) Keys() []string { return slices.Collect(maps.Keys(a)) }

// ao receber
ctx = otel.GetTextMapPropagator().Extract(ctx,
atributosSQS(msg.MessageAttributes))
ctx, span := otel.Tracer("cobranca.retorno").Start(ctx, "processar-arquivo",
    trace.WithSpanKind(trace.SpanKindConsumer),
    trace.WithAttributes(attribute.String("arquivo.id", id.String())))
defer span.End()
```

O resultado é o mesmo do curso de .NET: upload na API, fila, processamento e publicação do evento numa única linha do tempo, mesmo com a API e o worker em linguagens diferentes, porque o `traceparent` é um padrão W3C. Para pedir os atributos, o

`ReceiveMessage` precisa de `MessageAttributeNames: []string{"All"}`.

9.3 Logs

`log/slog` com handler JSON e o embrulho `comTrace` do curso de APIs (Módulo 11.3), que acrescenta `trace_id` a cada linha. As regras de volume são as do curso de .NET: nunca um log por linha do arquivo; um por arquivo e por lote, com os primeiros erros agregados. Use `log.LogAttrs(ctx, slog.LevelInfo, ...)` com atributos tipados (`slog.Int`, `slog.String`) no laço do worker: é a forma do `slog` que evita alocações quando o nível está ativo e não faz nada quando não está.

9.4 pprof e o perfil de vazamentos

Um servidor HTTP de administração em `127.0.0.1:6060` com `net/http/pprof` (o mesmo do curso de APIs, Módulo 9.1) dá acesso, com `kubectl port-forward`, a tudo que o `dotnet-counters`, o `dotnet-trace` e o `dotnet-gcdump` davam:

Pergunta	Perfil	Comando
Onde o tempo de CPU está indo?	<code>profile</code>	<code>go tool pprof -http=:8081 http://localhost:6060/debug/pprof/profile? seconds=30</code>
Quem está alocando?	<code>allocs</code>	<code>go tool pprof -sample_index=alloc_space .../debug/pprof/allocs</code>
O que está vivo no heap?	<code>heap</code>	<code>go tool pprof .../debug/pprof/heap</code>
O que cada goroutine está fazendo?	<code>goroutine</code>	<code>curl .../debug/pprof/goroutine?debug=2</code>
Há goroutines presas para sempre?	<code>goroutineleak</code> (Go 1.27)	<code>curl .../debug/pprof/goroutineleak?debug=1</code>
Onde as goroutines esperam por locks?	<code>mutex</code> , <code>block</code>	Exigem ligar a amostragem com <code>runtime.SetMutexProfileFraction</code> e <code>runtime.SetBlockProfileRate</code>

O perfil `goroutineleak` é o que mais combina com este curso. Um pipeline com o `select` esquecido do Módulo 5.8 deixa uma goroutine bloqueada num canal que ninguém mais vai ler; o perfil a encontra pelo GC, sem adivinhação. Rode-o depois de todo teste de carga.

As métricas do próprio runtime (heap, ciclos de GC, goroutines por estado, desde o Go 1.26) vêm do pacote `runtime/metrics`, e a instrumentação de runtime do OpenTelemetry as exporta para o mesmo backend das métricas de negócio.

9.5 Benchmarks

Para decisões de caminho quente, `testing.B` com `b.Loop()`:

```
func BenchmarkParseDetalhe(b *testing.B) {
    linhas := gerarLinhas(10_000)
    b.ReportAllocs()
    for b.Loop() {
        for i, l := range linhas {
            if _, _, ok := cnab400.ParseDetalhe(l, int32(i)); !ok {
                b.Fatal("linha inválida")
            }
        }
    }
}
```

`go test -bench ParseDetalhe -benchmem -count 10` roda dez vezes, e a ferramenta `benchstat` compara duas execuções (antes e depois de uma mudança) com análise estatística, que é o que o BenchmarkDotNet fazia sozinho. A coluna `allocs/op` deve ser zero para o parser.

9.6 Teste de carga e alertas

O teste de carga e os alertas são os do curso de .NET (Módulos 9.8 e 9.9): ambiente completo em Docker Compose, gerador de arquivos com a distribuição real de tamanhos, carga em degraus até encontrar o ponto em que a vazão para de subir, e alertas por sintoma (arquivo mais antigo na fila há mais de metade do SLA, DLQ não vazia, divergência de reconciliação, silêncio numa janela que deveria ter tráfego). Em Go, acrescente um: número de goroutines crescendo sem parar entre execuções, que é vazamento.

Exercício do Módulo 9

Instrumente o worker com as métricas de 9.1, incluindo o gauge de ocupação dos dois canais, e suba o Aspire Dashboard como coletor OTLP. Rode o teste de carga e identifique o gargalo só pelas métricas. Envie um arquivo pela API em Go e confirme que o trace atravessa API, fila e worker. Por fim, capture um perfil de CPU sob carga e outro de `allocs`, e confirme no flame graph que o parser não aparece entre as funções que alocam.